

Microsoft GraphRAG with an RDF Knowledge Graph - Part 2

Source: <https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-2-d8d291a39ed1>

Published: 2024-08-17T16:56:42Z

PDF generated from reader view on 2026-06-27 16:01 UTC

Uploading the output from Microsoft's GraphRAG into an RDF Store

Image 1: Ian Ormesher

(https://medium.com/@ianormy?source=post_page---byline--d8d291a39ed1-----)

4 min read

Aug 17, 2024

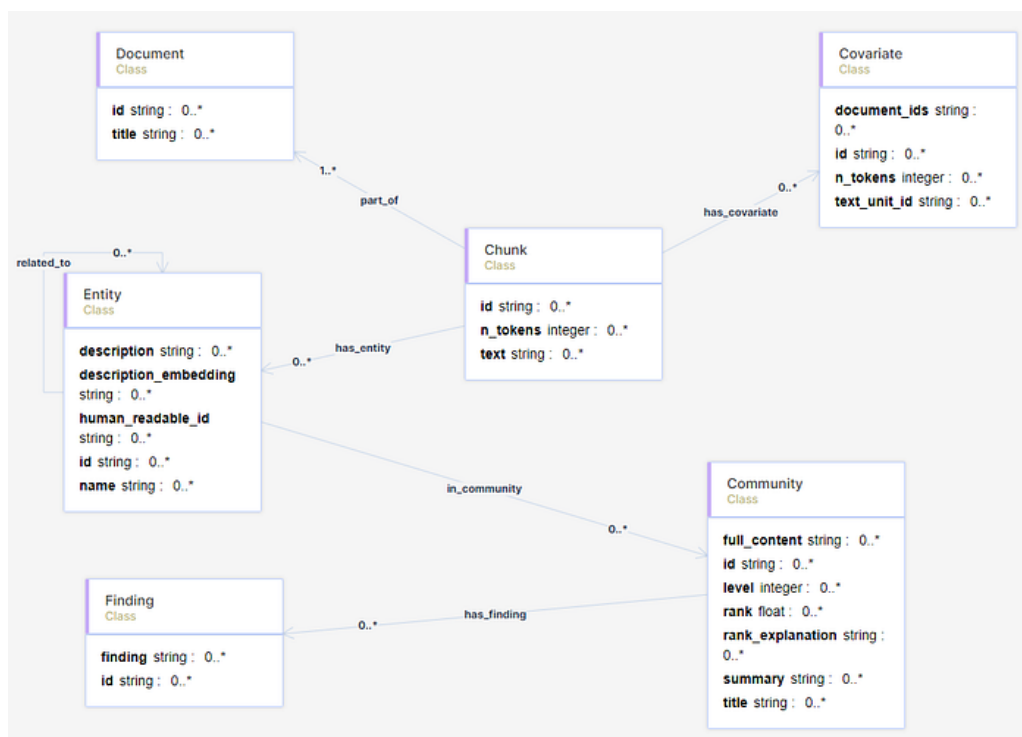


Image 2

Microsoft GraphRAG ontology

If you've followed the steps in Part 1

(<https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-1-00a354afdb09>) of this series then you should hopefully have some parquet files that were created. We're going to take those parquet files and transform them into a turtle format file to describe our linked data. You can read about the turtle format here (<https://linkeddata.github.io/rdflib.js/Documentation/turtle-intro.html>). We'll then import that turtle file into an RDF store. But before we import that file we'll import an ontology file I've created to describe the Microsoft GraphRAG. We'll also create a vector index that we can use for our semantic searching. Having done all that we'll be in a position to do RAG with the help of our RDF Knowledge Graph. That will be in Part 3. So, the steps in this Part are:

- Import Ontology into an RDF store
- Create and import data that fits the Ontology into the RDF store

- Create a vector index

Import Ontology into an RDF store

An ontology is a formal description of knowledge as a set of concepts within a domain and the relationships that hold between them. They are an important basis for Knowledge Graphs. You can read more about them here (<https://www.ontotext.com/knowledgehub/fundamentals/what-are-ontologies/>). The W3C have created a specification for a format that allows us to define them called OWL2 (<https://www.w3.org/TR/owl2-primer/>). I've created an ontology for the Microsoft GraphRAG that we'll be using and it looks like this:

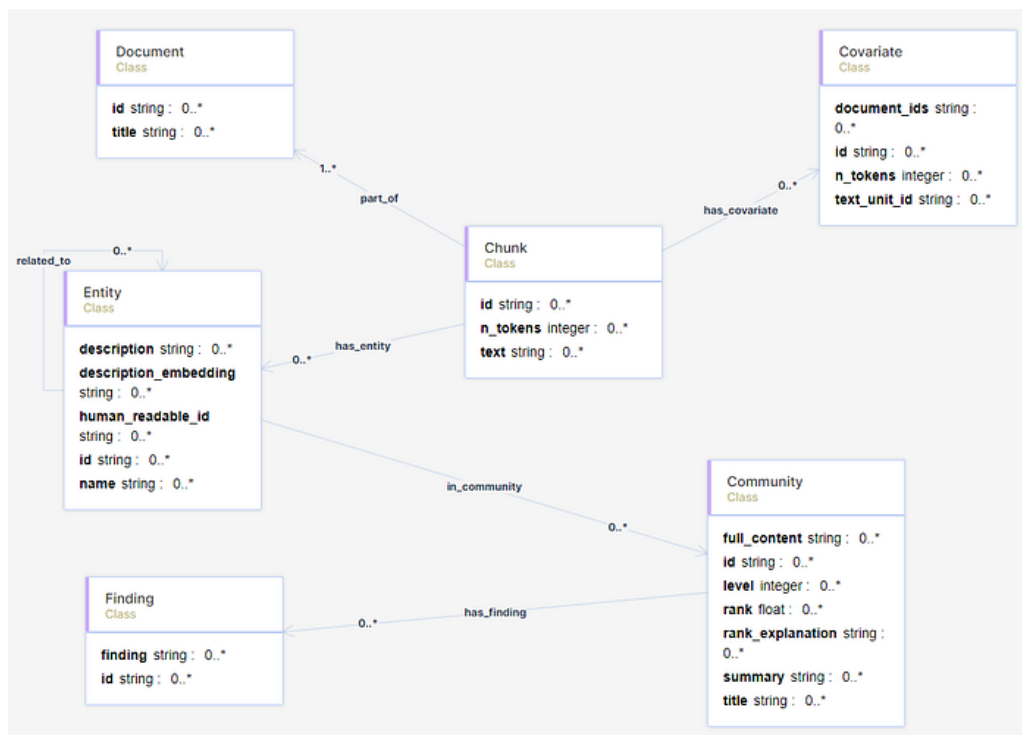


Image 3

Microsoft GraphRAG ontology

I created this ontology using a great product called Metaphactory (<https://metaphacts.com/>) and then exported it as an OWL file. Check the link in the Resources section to get a copy of this ontology file together with all the other files (and notebooks) related to this part.

Ontologies are an important part of Knowledge Graphs. They allow for the definition of the data and the relationships in a standardised way. They also provide the ability to validate the data and to explain it. It's really important that the systems we create aren't simply black boxes that no-one can understand. We need to be able to explain our systems and outputs and verify our data that is input.

Get Ian Ormesher's stories in your inbox

Join Medium for free to get updates from this writer.

Remember me for faster sign in

First we'll create an empty repository in an RDF store and then we'll import the ontology file. I'm using GraphDB (<https://www.ontotext.com/products/graphdb/download/>) as my RDF store (since they provide a free version), so you would create a new repository (I named it msft-graphrag) like this:

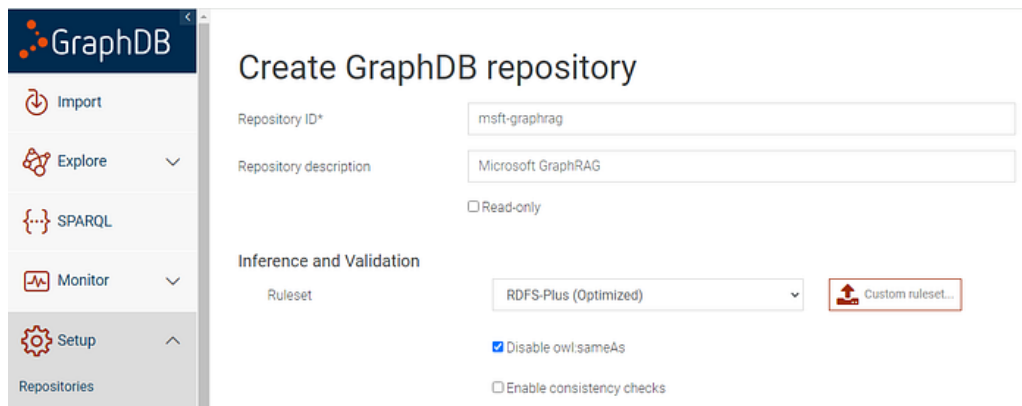


Image 4

Create repository in GraphDB

Once you've created the repository you can then import the ontology OWL file I've provided:

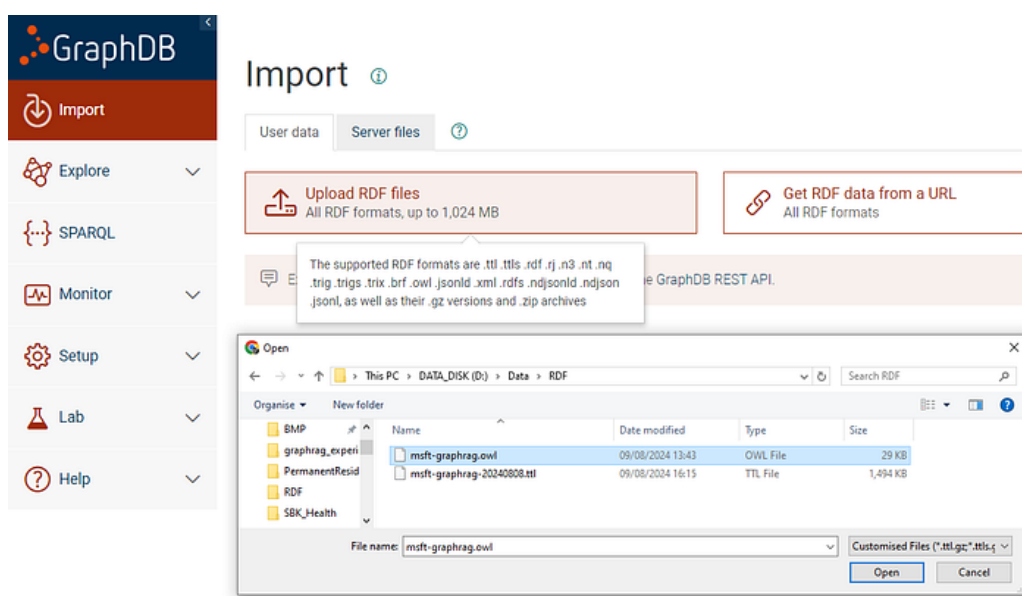


Image 5

import ontology into the repository

When you import it, make sure you specify "The default graph":

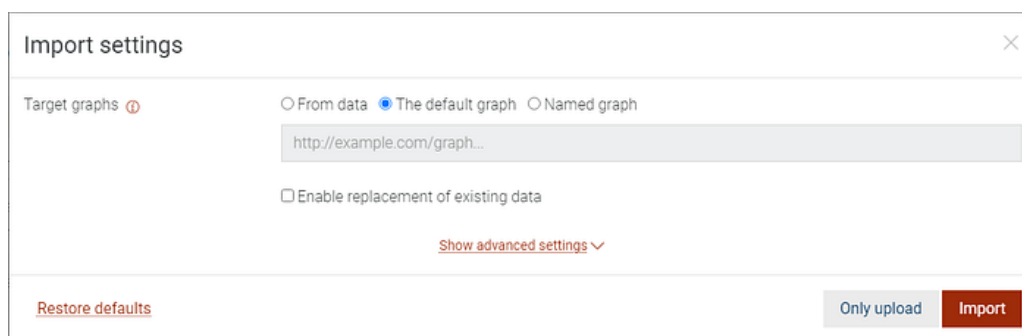


Image 6

import into the default graph

Create and import data that fits the Ontology into the RDF store

Now we've imported the ontology into the RDF store we are safe to create and import the data.

I have created a Jupyter notebook that reads in the parquet files produced as output from Part 1 and then creates a corresponding turtle file that can be imported into the graph. If all you want to do is import the data without wondering how it was created, I have also provided 2 turtle files that you can use. One that was done with 300 chunk sizes, and the other that was done with 1200 chunk size and correlation data. Checkout the resources at the end for those files.

I have also created another notebook to analyse the data that is now in our Knowledge Graph. That's also on my Github.

Create a Vector Index

In the last part of my Jupyter notebook I show how to create a vector index for the data using Elasticsearch (<https://www.elastic.co/elasticsearch/vector-database>). There is a free version of this product that you can use which you can download from here (<https://www.elastic.co/downloads/elasticsearch>).

We'll create an index for the `Entity` class, storing the **`description_embedding`** field. This field is in the parquet file for the entities so we'll tie it to the `id` field.

Having created this index we will use it in Part 3 (<https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-3-328f85d7dab2>), together with our RDF Knowledge Graph, to generate better responses to users questions.